Báo cáo bài tập lớn

Môn: Nhập môn trí tuệ nhân tạo

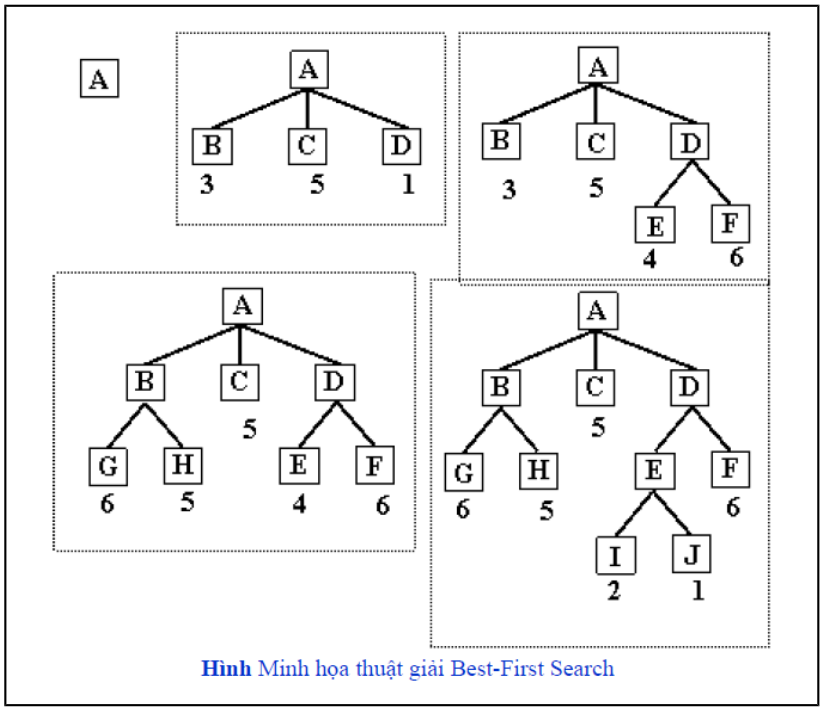
<u>Đề bài:</u> Áp dụng thuật toán best first search vào tìm đường đi từ một điểm đến một điểm khác trong bản đồ của một xã

<u>A. Thuật toán best first search:</u>

Ưu điểm của tìm kiếm theo chiều sâu là không phải quan tâm đến sự mở rộng của tất cả các nhánh. Ưu điểm của tìm kiếm chiều rộng là không bị sa vào các đường dẫn bế tắc (các nhánh cụt). Tìm kiếm ưu tiên tối ưu sẽ kết hợp 2 phương pháp trên cho phép ta đi theo một con đường duy nhất tại một thời điểm, nhưng đồng thời vẫn "quan sát" được những hướng khác. Nếu con đường đang đi "có vẻ" không triển vọng bằng những con đường ta đang "quan sát" ta sẽ chuyển sang đi theo một trong số các con đường này. Để tiện lợi ta sẽ dùng chữ viết tắt BFS thay cho tên gọi tìm kiếm ưu tiên tối ưu.

Một cách cụ thể, tại mỗi bước của tìm kiếm BFS, ta chọn đi theo trạng thái có khả năng cao nhất trong số các trạng thái được được xét *cho đến thời điểm đó.* (khác với leo đồi dốc đứng là chỉ chọn trạng thái có khả năng cao nhất trong số các trạng thái kế tiếp có thể đến được từ trạng thái hiện tại). Như vậy, với tiếp cận này, ta sẽ ưu tiên đi vào những nhánh tìm kiếm có khả năng nhất (giống tìm kiếm leo đồi dốc đứng), nhưng ta sẽ không bị lẩn quẩn trong các nhánh này vì nếu càng đi sâu vào một hướng mà ta phát hiện ra rằng hướng này càng đi thì càng tệ, đến mức nó xấu hơn cả những hướng mà ta chưa đi, thì ta sẽ không đi tiếp hướng hiện tại nữa mà chọn đi theo một hướng tốt nhất trong số những hướng chưa đi. Đó là tư tưởng chủ đạo của tìm kiếm BFS.

Ta xét ví dụ sau:



Hình Minh họa thuật giải Best-First Search

Khởi đầu, chỉ có một nút (trạng thái) A nên nó sẽ được mở rộng tạo ra 3 nút mới B,C và D. Các con số dưới nút là giá trị cho biết độ tốt của nút. Con số càng nhỏ, nút càng tốt. Do D là nút có khả năng nhất nên nó sẽ được mở rộng tiếp sau nút A và sinh ra 2 nút kế tiếp là E và F. Đến đây, ta lại thấy nút B có vẻ có khả năng nhất (trong các nút B,C,E,F) nên ta sẽ chọn mở rộng nút B và tạo ra 2 nút G và H. Nhưng lại một lần nữa, hai nút G, H này được đánh giá ít khả năng hơn **E**, v. thế sự chú. lại trở về E. E được mở rộng và các nút được sinh ra từ E là I và J. Ở bước kế tiếp, J sẽ được mở rộng và nó có khả năng nhất. Quá trình này tiếp tục cho đến khi tìm thấy một lời giải.

Lưu ý rằng tìm kiếm này rất giống với tìm kiếm leo đồi dốc đứng, với 2 ngoại lệ. Trong leo núi, một trạng thái được chọn và tất cả các trạng thái khác bị loại bỏ, không bao giờ chúng được xem xét lại. Cách xử lý dứt khoát này là một đặc trưng của leo đồi. Trong BFS, tại một bước, cũng có một di chuyển được chọn nhưng những cái khác vẫn được giữ lại, để ta có thể trở lại xét sau đó khi trạng thái hiện tại trở nên kém khả năng hơn những trạng thái đ. được lưu trữ. Hơn nữa, ta chọn trạng thái tốt nhất mà không quan tâm

đến nó có *tốt hơn* hay không các trạng thái trước đó. Điều này tương phản với leo đồi vì leo đồi sẽ dừng nếu không có trạng thái tiếp theo nào tốt hơn trạng thái hiện hành.

Để cài đặt các thuật giải theo kiểu t.m kiếm BFS, người ta thường cần dùng 2 tập hợp sau

OPEN: tập chứa các trạng thái đã được sinh ra nhưng chưa được xét đến (vì ta đã chọn một trạng thái khác). Thực ra, **OPEN** là một loại hàng đợi ưu tiên (priority queue) mà trong đó, phần tử có độ ưu tiên cao nhất là phần tử *tốt nhất*. Người ta thường cài đặt hàng đợi ưu tiên bằng Heap. Các bạn có thể tham khảo thêm trong các tài liệu về Cấu trúc dữ liệu về loại dữ liệu này.

CLOSE: tập chứa các trạng thái đã được xét đến. Chúng ta cần lưu trữ những trạng thái này trong bộ nhớ để đề ph.ng trường hợp khi một trạng thái mới được tạo ra lại trùng với một trạng thái mà ta đã xét đến trước đó. Trong trường hợp không gian tìm kiếm có dạng cây thì không cần dùng tập này.

Thuật giải BEST-FIRST SEARCH

1. Đặt **OPEN** chứa trạng thái khởi đầu.

2. Cho đến khi t.m được trạng thái đích hoặc không c.n nút nào trong OPEN, thực hiện:

2.a. Chọn trạng thái **tốt nhất** (Tmax) trong OPEN (và xóa Tmax khỏi OPEN)

2.b. Nếu Tmax là trạng thái kết thúc th. thoát.

2.c. Ngược lại, tạo ra các trạng thái kế tiếp Tk có thể có từ trạng thái Tmax. Đối với mỗi trạng thái kế tiếp Tk thực hiện:

Áp dụng vào bài toán:

Khai báo chương trình:

```
private int n;//so dinh
        private int[,] arr = new int[50, 50];//luu do thi mang 2 chieu
        private int[] head = new int[50];//head là khi tìm đc đường thì
head[i] = j là đến j là đỉnh trc của i
        private int first, last, begin, end;
        private int[] queue = new int[50];
        private int[] dd = new int[50];// dd là đánh dấu các đỉnh đa
duyệt
        private int[] h = new int[50];
        private string path = "";
```

thực hiện chương trình:

```
public void Process()
        {
            for (int i = 0; i < n; i++)
            {
                head[i] = dd[i] = -1;
```

```
            }
            first = 0;
            last = 0;
            queue[last] = begin;
            dd[begin] = 0;
            while (first <= last)
            {
                int i = queue[first];
                first++;
                if (i == end) break;
                else
                {
                    for(int j = 0; j < n; j++)
                        if (dd[j] == -1 && arr[i, j] > 0)
                        {
                            last++;
                            queue[last] = j;
                            int k = last;
                            while (k > first)
                            {
                                if (h[k] < h[k - 1])
                                    {
                                        int kk = h[k+1];
                                        h[k + 1] = h[k];
                                        h[k] = kk;
                                        k--;
                                    }
                                else break;

                            }
                            head[j] = i;
                            dd[j] = 0;
                        }
                }
            }
            path = "";
            if (head[end] != -1)
            {
                path = (char)(end + 65) + path;
                end = head[end];
                while (head[end] != -1)
                {
                    path = (char)(end + 65) + " --> " + path;
                    end = head[end];
                }
                path = (char)(end + 65) + " --> " +path;
            }
            else path = "null";
        }
```

Tài liệu được tham khảo ở một số web trên mạng.